DSPy 3 + GEPA: A estrutura RAG mais avançada até agora — Raciocínio automático e sugestões

Tiago Cassiano d'Avila
em Recursos de Aprendizado de IA
Publicado em 2025-08-25

Pressione Enter ou clique para ver a imagem em tamanho real

Na semana passada, o OpenAI soltou um aumento de empilhamento de acessos no meio da sua equipe de pesquisa. Não foi GPT-5 ou Sora, foi o trabalho de Kevin Zhang e Omar Khattab e colegas sobre **DSPy + GEPA**, uma estrutura de IA que ultrapassa o ajuste de prompt e adiciona **raciocínio automático e sugestões** para sistemas de IA em produção.

Para quem não está familiarizado, o DSPy é a estrutura de linguagem declarativa de Stanford para otimizar os prompts e pesos de modelos de linguagem. Em vez de escrever prompts longos e frágeis à mão, você define módulos e assinaturas, e o DSPy compila automaticamente exemplos e instruções para melhorar o desempenho. O GEPA (Genetic-Pareto Prompt Optimizer) é o novo otimizador que adiciona reflexão em linguagem natural a essa pilha. Ele aprende com rastreamentos de execução e propõe melhorias em nível de arquitetura, não apenas ajustes de prompt.

Os ganhos são significativos: o GEPA supera o GRPO em **+6% em média e até +20%** em alguns casos, usando **35 vezes menos rollouts**. O DSPy também inclui operadores de nível superior como `MultiChainComparison` (amostragem de candidatos + comparação entre eles) e `ProgramOfThought` (decomposição de tarefas em etapas lógicas), que se compõem bem com o GEPA para tarefas de raciocínio mais complexas.

Neste artigo, vamos construir um sistema RAG de ponta a ponta usando DSPy 3 + GEPA, avaliá-lo no HotPotQA, salvar os artefatos compilados e observar o otimizador aprender a partir de falhas. Tudo isso em menos de 50 linhas de código. Vamos lá.

O que há de novo no DSPy 3 / GEPA

- **Reflexão em linguagem natural:** o GEPA lê os rastreamentos de execução e propõe melhorias em nível de arquitetura, não apenas ajustes de prompt.
- **Operadores de nível superior:** `MultiChainComparison` amostra candidatos e os compara; `ProgramOfThought` decompõe tarefas em etapas lógicas.
- **Compatível com o otimizador:** os operadores se compõem com o GEPA e otimizadores MIPROv2 para pipelines de produção.
- **Salvar/carregar artefatos compilados:** `dspy.save("path")` e `dspy.load("path")` persistem programas otimizados para implantação.

Instalação e configuração

```
pip install dspy
export OPENAI_API_KEY=sk-...
export ANTHROPIC_API_KEY=sk-ant-...
export GOOGLE_API_KEY=...
export OPENROUTER_API_KEY=...
export TOGETHER_API_KEY=...
export DEEPSEEK_API_KEY=...
export GROQ_API_KEY=...
export CEREBRAS_API_KEY=...
export OLLAMA_API_BASE=http://localhost:11434
export LM_STUDIO_API_BASE=http://localhost:1234
export OPENAI_COMPATIBLE_API_BASE=http://your-endpoint:8000
export OPENAI_COMPATIBLE_API_KEY=optional
```

Configure o modelo e o coletor de dados

```
import dspy, os, random, collections, statistics, time, pathlib
from dspy.datasets import HotPotQA

# Configure LM via OpenRouter (swap for any provider you like)
lm = dspy.LM("openrouter/google/gemini-2.5-flash", api_key=os.environ["OPENROUTER_API_KEY"])
dspy.settings.configure(lm=lm)

# Load HotPotQA: 2 hops, retrieval-augmented
dataset = HotPotQA(train_seed=1, train_size=500, eval_seed=2023, dev_size=200, test_size=0)
trainset = [x.inputs() for x in dataset.train]
devset = [x.inputs() for x in dataset.dev]

print(f"Loaded {len(trainset)} train, {len(devset)} dev")
```

Construa o programa RAG

```
class GenerateSearchQuery(dspy.Signature):
    """Gera uma consulta de pesquisa otimizada para recuperar fatos adicionais."""
    context = dspy.InputField(desc="fatos já coletados")
    question = dspy.InputField()
    query = dspy.OutputField(desc="consulta de pesquisa")

class SelfRAG(dspy.Module):
    def __init__(self, passages=10):
        super().__init__()
        self.generate_query = [dspy.ChainOfThought(GenerateSearchQuery), dspy.ChainOfThought(GenerateSearchQuery)]
        self.retrieve = dspy.Retrieve(k=passages)
        self.generate_answer = dspy.ChainOfThought("context, question -> answer")
    def forward(self, question):
        context = []
        for hop in range(2):
            q = self.generate_query[hop](context=context, question=question).query
            passages = self.retrieve(q).passages
            context = list(set(context + passages))
        return self.generate_answer(context=context, question=question)

print("Programa RAG definido")
```

Métrica + coletor de falhas

```
def exact_match(example, prediction, trace=None):
    from dspy.evaluate import EM
    return EM(example.answer, prediction.answer)

failures = []
def collect_failures(example, prediction, trace=None):
    from dspy.evaluate import EM
    if not EM(example.answer, prediction.answer):
        failures.append(dict(question=example.question, gold=example.answer, predicted=getattr(prediction, "answer", "")))
    return True

print("Métrica + coletor de falhas prontos")
```

Avaliação de linha de base

```
from dspy.evaluate import Evaluate
ev = Evaluate(devset=devset[:50], metric=exact_match, num_threads=8, display_progress=True)
baseline = ev(SelfRAG())
print(f"Linha de base: {baseline.score:}")
```

Compilar com GEPA

```
import inspect
kwargs = dict(metric=exact_match, auto="light", num_threads=8, reflection_lm=lm, track_stats=True)
if "track_stats" in inspect.signature(dspy.GEPA.compile).parameters: kwargs["track_stats"] = True
gepa = dspy.GEPA(**kwargs)
compiled = gepa.compile(student=SelfRAG(), trainset=trainset[:200], valset=devset[:100])
print("Compilado com GEPA")
```

Avaliar e salvar

```
baseline.save("results/baseline.jsonl")
ev(compiled).save("results/gepa.jsonl")
compiled.save("artifacts/self_rag_gepa.v1")
print("Salvo em results/ e artifacts/")
```

Inspecionar o que o GEPA aprendeu

```
for name, p in compiled.named_predictors():
    print(f"{name}\n{p.extended_signature.instructions}\")
```

Ler os rastreamentos de falhas

```
import pathlib, collections
for ln in pathlib.Path("results/gepa.jsonl").open:
    ...
```

Por que isso importa

O RAG ainda é difícil em produção. Recuperação de uma vez, cadeias frágeis, sem ciclo de aprendizado a partir de falhas. O DSPy + GEPA muda isso: você define módulos e métricas, executa o compilador e implanta artefatos que melhoram automaticamente. Os rastreamentos de falhas alimentam a próxima rodada de otimização. Não é mágica, é engenharia de sistemas para IA.

Experimente o exemplo de 50 linhas, substitua o conjunto de dados pelo seu domínio, e observe o otimizador aprender a partir dos seus casos específicos. Os artefatos compilados se compõem com o restante da sua pilha de IA: LangGraph para orquestração, MCP para ferramentas, OpenTelemetry para observabilidade.

Se você está construindo sistemas de IA que precisam melhorar sem reescrever prompts manualmente, o DSPy + GEPA é o caminho a seguir.

Antes de começar

Se você chegou até aqui, provavelmente você se interessa por IA aplicada, engenharia de prompt ou avaliação de LLMs. Se isso ressoa com o seu trabalho, eu adoraria ouvir sobre o que você está construindo. Sinta-se à vontade para me seguir ou entrar em contato por meio do LinkedIn ou GitHub.

Se você quer aprofundar-se em sistemas de IA em produção, avaliação de agentes ou otimização de LLMs, estou ministrando cursos sobre isso. Também ajudo equipes a implantar esses sistemas na prática. Se isso for útil para o seu time, entre em contato.

Leituras e recursos complementares

- Artigo do GEPA: "Reflective Prompt Evolution Can Outperform Reinforcement Learning" (arXiv 2507.19457)
- Artigo do MIPROv2: "Scaling Prompt Optimization with Bayesian Optimization" (arXiv 2406.11695)
- Artigo do LangProBench: "Language Models Ought to Know More Than Their Prompts" (arXiv 2502.18470)
- Documentação do DSPy 3: dspy.ai
- Repositório do GEPA: github.com/gepa-ai/gepa

Tiago Cassiano d'Avila — Engenheiro de IA aplicada focado em sistemas de produção. Construindo ferramentas de avaliação e otimização para LLMs.